C Structures & the GPIOx Peripheral

From a C `struct` to a Physical Pin

Professor: Nerio Andrés Montoya
`namontoy@unal.edu.co`

2026-S2

Universidad Nacional de Colombia

Agenda

C Structures

SFR — Special Function Registers

The GPIOx Peripheral

C Meets Hardware

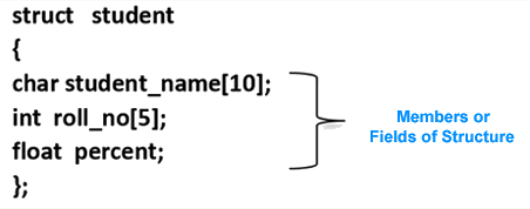
Closing

C Structures

What is a Structure?

A structure is a user-defined data type in C that combines variables of different types under one name.

- Similar to an array, but an array holds items of the *same* type
- A struct can group *different* types together — exactly what a real peripheral needs



Structure Examples

```
typedef struct
{
    char        ID;
    uint8_t     repetitions;
    uint8_t     dummy;
    int16_t     counterUp;
    uint32_t    timestamp;
} BasicExample;
```

Structures & Pointers

```
typedef struct {
    uint32_t x;
    uint32_t y;
} Point;

Point p1 = {10, 20};
Point *ptr = &p1;

ptr->x = 15;          // arrow operator: dereference + access
   field
(*ptr).y = 25;         // equivalent, but arrow is the idiom
```

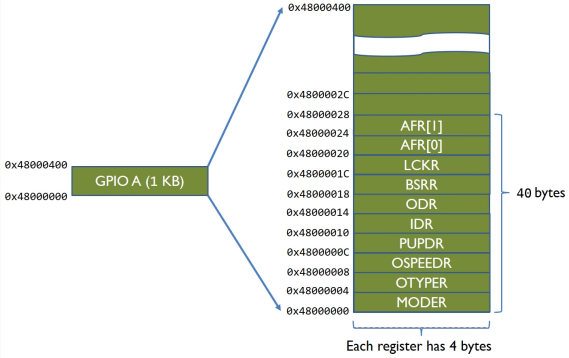
- `->` is shorthand for “follow the pointer, then get this field”
- This is exactly the syntax you’ll use to touch every peripheral register from now on

SFR — Special Function Registers

What is an SFR?

- A **Special Function Register** is a small block of memory that isn't RAM at all — it's a direct handle on a piece of hardware
- Writing to it changes something physical (a pin's voltage); reading it reports something physical (is a button pressed?)
- Each SFR lives at a **fixed memory address**, fixed by the chip manufacturer — documented in the Reference Manual

Memory-Mapped I/O



- RAM, Flash, and peripheral registers all share *one* address space — there's no special "I/O instruction" in ARM Cortex-M, just ordinary reads and writes
- If you know a peripheral's base address, a plain C pointer reaches it

From Address to Struct

```
typedef struct {
    volatile uint32_t MODER;    // offset 0x00
    volatile uint32_t OTYPER;   // offset 0x04
    volatile uint32_t OSPEEDR;  // offset 0x08
    volatile uint32_t PUPDR;    // offset 0x0C
    volatile uint32_t IDR;      // offset 0x10 (read-only)
    volatile uint32_t ODR;      // offset 0x14
    volatile uint32_t BSRR;     // offset 0x18
} GPIO_TypeDef;

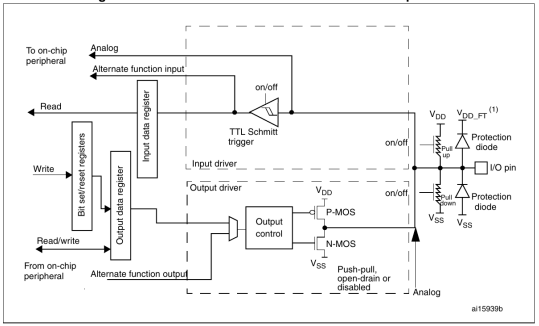
#define GPIOA ((GPIO_TypeDef *) 0x40020000)
```

- This is (a simplified version of) what STM32's CMSIS headers actually declare — `GPIOA->ODR` is just a named offset from a hard-coded address

The GPIOx Peripheral

GPIOx Internal Configuration

Figure 16. Basic structure of a five-volt tolerant I/O port bit



1. V_{DD_FT} is a potential specific to five-volt tolerant I/Os and different from V_{DD}.

The Structure Applied to GPIOx

```
typedef struct
{
    volatile uint32_t MODER;    // port mode register                    ADDR_OFFSET:    0x00
    volatile uint32_t OTYPER;   // port output type register             ADDR_OFFSET:    0x04
    volatile uint32_t OSPEEDR;  // port output speed register            ADDR_OFFSET:    0x08
    volatile uint32_t PUPDR;    // port pull-up/pull-down register       ADDR_OFFSET:    0x0C
    volatile uint32_t IDR;      // port input data register              ADDR_OFFSET:    0x10
    volatile uint32_t ODR;      // port output data register             ADDR_OFFSET:    0x14
    volatile uint32_t BSRR;     // port bit set/reset register           ADDR_OFFSET:    0x18
    volatile uint32_t LCKR;     // port configuration lock register      ADDR_OFFSET:    0x1C
    volatile uint32_t AFRL;     // alternate function low register       ADDR_OFFSET:    0x20
    volatile uint32_t AFRH;     // alternate function high register      ADDR_OFFSET:    0x24
} GPIOx_RegDef_t;
```

- Every box in this diagram is one `volatile uint32_t` field, at a fixed offset from `GPIOA`'s base address

GPIO Registers, One by One

Register	What it controls
`MODER`	Pin mode: input / output / alternate function / analog
`OTYPER`	Output type: push-pull or open-drain
`OSPEEDR`	Output slew rate (switching speed)
`PUPDR`	Internal pull-up / pull-down
`IDR`	*Input* data — read-only, current pin state
`ODR`	*Output* data — what you're driving the pin to
`BSRR`	Atomic bit Set/Reset — the safe way to change one pin

C Meets Hardware

Pointers: The Bridge to Hardware

```
uint32_t *reg = (uint32_t *) 0x40020014;

*reg = 0x1;              // write 1 to that address
uint32_t val = *reg;     // read it back
```

- A pointer *holds a memory address*
- `*` dereferences it — “go to that address”
- Peripheral registers *are* fixed memory addresses



Bitwise Operators: Talking to Registers

```c
#define LED_PIN (1U << 5)

GPIOA->ODR |=  LED_PIN;   // SET bit 5    -> LED on
GPIOA->ODR &= ~LED_PIN;   // CLEAR bit 5  -> LED off
GPIOA->ODR ^=  LED_PIN;   // TOGGLE bit 5
```

ODR before:	0	0	0	0	0	0	0	0
after SET bit 5:	0	0	0	0	0	1	0	0

Structs & `volatile`: How HAL Sees a Peripheral

```c
typedef struct {
    volatile uint32_t MODER;
    volatile uint32_t OTYPER;
    volatile uint32_t ODR;
} GPIO_TypeDef;

#define GPIOA ((GPIO_TypeDef *) 0x40020000)

GPIOA->ODR |= (1U << 5);   // same idea, friendlier syntax
```

- A `struct` overlays field names onto a block of memory — this *is* what STM32's HAL headers do
- `volatile` tells the compiler: "don't optimize this away — the value can change outside program flow" (an interrupt, or the hardware itself)

Reading and Writing a Pin

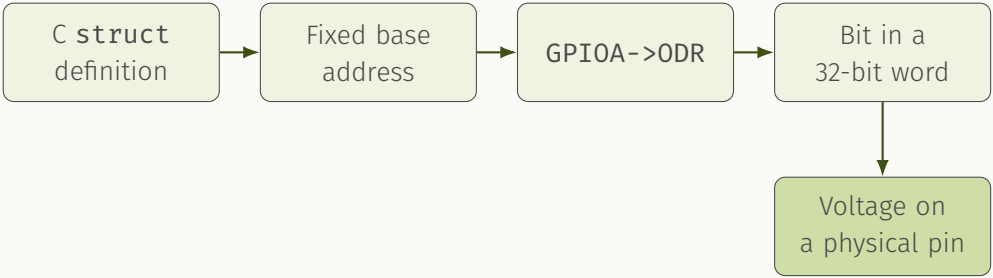
```
GPIOA->ODR |= (1U << 5);      // set pin 5 -- works, but not
    atomic

GPIOA->BSRR = (1U << 5);      // set pin 5 -- atomic, preferred
GPIOA->BSRR = (1U << (5+16)); // clear pin 5 (upper 16 bits =
    reset)

if (GPIOA->IDR & (1U << 0)) { // read pin 0
    // button is high
}
```

- `ODR |=` is a read-modify-write — an interrupt between the read and the write can corrupt other bits
- `BSRR` changes exactly one bit in a single write, with no such race

The Full Chain



- Every layer we've covered — structs, memory addresses, registers, bitwise operators — exists to make this last arrow happen reliably

Application Example

Struct

Basic example working with structures — using the normal definition, and then using pointers to reach the same fields.

- Live coding: declare a small struct, access it directly and through a pointer, then apply the same idea to `GPIOA`

Closing

Key Takeaways

- A struct groups related variables under one name — an SFR block *is* a struct, just placed at a fixed address
- Cortex-M has no separate I/O instructions: peripherals are reached with ordinary pointers, into ordinary memory space
- `GPIOA->ODR` is real C, not magic — a named offset from a hard-coded base address
- Prefer `BSRR` over `ODR |=` when setting or clearing a single pin — it's atomic